# Trustworthy Fit/Gap Report

## 1. Product Context

AI Interview helps assessors, recruiters, and hiring managers evaluate candidates from structured interview evidence. The candidate does not choose this assessment flow, so the report must be transparent, careful with uncertainty, and useful for human review instead of acting like an opaque verdict.

Claimed depth: frontend-depth fullstack slice. Backend changes are intentionally additive to stabilize the API contract; the main craft focus is the assessor-facing report experience.

Repository branch:

- Fork branch: `https://github.com/samudrazulqifli/ai-interview-platform/tree/frontend-depth-fitgap-report`
- Pull Request creation URL: `https://github.com/rakamindev/ai-interview-platform/compare/main...samudrazulqifli:ai-interview-platform:frontend-depth-fitgap-report?expand=1`
- Pull Request link: pending final GitHub submission URL after creating the PR from the fork branch.

## 2. Core Problem

The existing Fit/Gap Report compares candidate skill levels with vacancy requirements, but it reads like a raw table. It does not clearly answer the assessor's decision questions:

- Is this candidate ready enough to proceed?
- Which gaps are material?
- Which requirements were not assessed?
- Which signals are low confidence or manually overridden?

There is also a backend/frontend contract mismatch: the API stores expected_level, while the frontend reads required_level. That can make required skill levels appear blank or unreliable in the UI.

## 3. Severity-Ranked Findings

P1 - Required level contract mismatch harms trust in the report. The backend emits expected_level, while the frontend expects required_level, so the comparison table may fail to show role requirements correctly.

P1 - No decision summary or risk signal. Assessors must manually infer fit from row-level data, increasing the chance of advancing a candidate with unassessed or critical-gap skills.

P2 - Unknown evidence is under-emphasized. not_assessed skills are a major hiring risk but appear as just another table row.

P2 - Low-confidence and human override signals are not prominent. Reviewers need to know when AI evidence is weak or when a human changed the model's level.

P3 - Loading, generating, unavailable, and long-text states are basic. The page works, but it does not feel like a polished decision-support surface.

## 4. Acceptance Criteria

- The API returns both `expected_level` and `required_level` for backward-compatible skill comparisons.
- The API returns deterministic `comparison_summary` metadata.
- The report shows readiness score, risk level, recommendation, and review flags.
- The table displays required level, candidate level, confidence, result, low-confidence state, not-assessed state, and human override state.
- The UI handles loading, generating, unavailable report, long narrative, discovered skills, and responsive layouts.
- The feature remains usable even if an older report does not include `comparison_summary`.
- Unit tests cover the deterministic summary behavior.

## 5. Options And Trade-Offs

Option A - Frontend-only derived summary.
Fastest and very UI-focused, but every consumer would need to reimplement summary logic and exported JSON/PDF would

not share a stable contract.

Option B - Backend deterministic summary plus frontend dashboard.
Chosen. Slightly more backend work, but it creates a stable API contract and lets the frontend focus on presentation, states, and review workflow.

Option C - Ask Gemini to produce the whole hiring recommendation.
Richer natural language, but risky for a high-impact candidate decision because model output is less deterministic and harder to test.

## 6. Implementation Summary

- Added `required_level` alias while retaining `expected_level`.
- Added `is_override` to skill comparisons.
- Added `FitGap::Summary` for readiness score, risk level, counts, recommendation, and review flags.
- Added `comparison_summary` to the fit/gap API response and JSON export path.
- Reworked the Fit/Gap Report page into a compact decision dashboard.
- Improved the skill comparison table with confidence, not-assessed highlighting, critical-gap highlighting, and override visibility.
- Added RSpec coverage for summary behavior.

## 7. Verification Notes

Automated checks:

cd api
bundle exec rspec spec/services/fit_gap/summary_spec.rb

cd ../web
npm run build

Manual verification performed on local environment:

- Local frontend URL: `http://localhost:5173/assessments/1/sessions/1/fitgap/1`
- Candidate/session context: assessment `1`, session `1`, vacancy `1`
- Fit/Gap report generated successfully and rendered the decision dashboard.
- Decision Readiness displayed `42 / 100`.
- Risk badge displayed `High Risk`.
- Required skill coverage displayed `2 of 3 required skills assessed`.
- Fit Signals displayed `Match: 0` and `Exceeds: 0`.
- Gaps displayed `Any gap: 2` and `Critical: 1`.
- Unknowns displayed `Not assessed: 1` and `Low confidence: 1`.
- Assessor recommendation displayed: high-risk or insufficient fit signal; re-assess flagged skills before advancing.
- Review flags displayed:
- 1 required skill(s) not assessed
- 1 critical gap(s) of two or more levels
- 1 low-confidence assessed skill(s)
- Skill Comparison table displayed required level, candidate level, confidence, and result columns.
- React / Frontend Development Core displayed as `Required L4`, `Candidate L3`, `High confidence`, `Gap -1`.
- System Design & Architecture displayed as `Required L3`, `Not assessed`.
- TypeScript & Type Systems displayed as `Required L4`, `Candidate L2`, `Low confidence`, `Gap -2`.

Seeded fault verification:

- Temporary injected fault: changed critical-gap threshold from `delta <= -2` to `delta <= -3`.
- Expected failure occurred in `FitGap::Summary flags missing and critical skill evidence as high risk`.
- Failure proved the test catches a two-level critical gap regression: expected `critical_gap_count` to equal `1`, got `0`.
- The threshold was restored to `delta <= -2`.
- `bundle exec rspec spec/services/fit_gap/summary_spec.rb` passed again with `3 examples, 0 failures`.

Seeded fault test idea: temporarily change critical_gap_count to count only delta <= -3; the high-risk summary spec should fail for a two-level gap.

AI verification note: AI initially suggested adding frontend-only summary logic. I kept a frontend fallback for backward compatibility, but moved the source of truth into a tested backend summary so export/API consumers receive the same decision metadata.

## 8. Screenshots To Capture

Embedded visual evidence:

- `rspec-summary-success.png` - backend summary unit test success.
- `web-build-success.png` - frontend TypeScript and Vite production build success.
- `fitgap-dashboard-desktop.png` - Fit/Gap report dashboard with readiness score, high-risk badge, summary cards, review flags, and skill comparison table.
- `fitgap-narrative-discovered-skills.png` - narrative section and discovered skills evidence.

Recommended follow-up screenshots before final portal submission:

- `fitgap-mobile.png` - mobile responsive view.
- `seeded-fault-fail.png` - seeded fault test failure.
- `seeded-fault-fixed.png` - seeded fault restored to passing.

## 9. Video Demo Outline

1. Show the candidate portfolio and vacancy context.
2. Open Fit/Gap Report.
3. Explain readiness score, risk badge, recommendation, and review flags.
4. Walk through the comparison table.
5. Show export buttons and responsive layout.

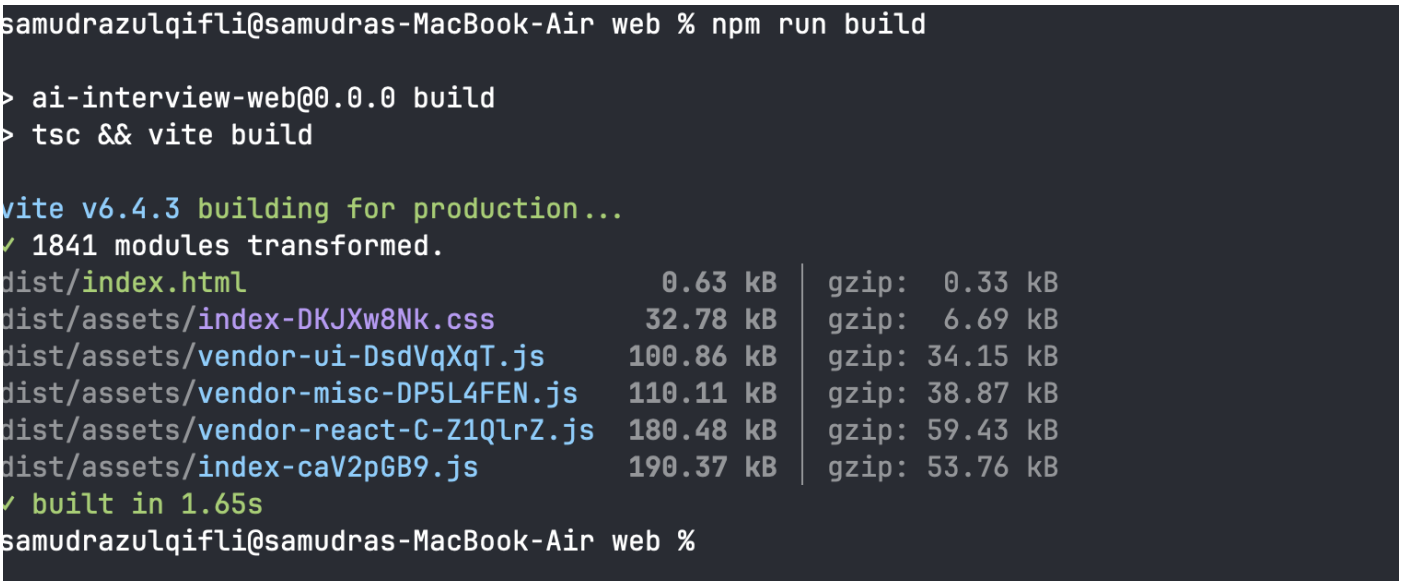
# Visual Evidence

**Figure 1. Backend summary unit test success**

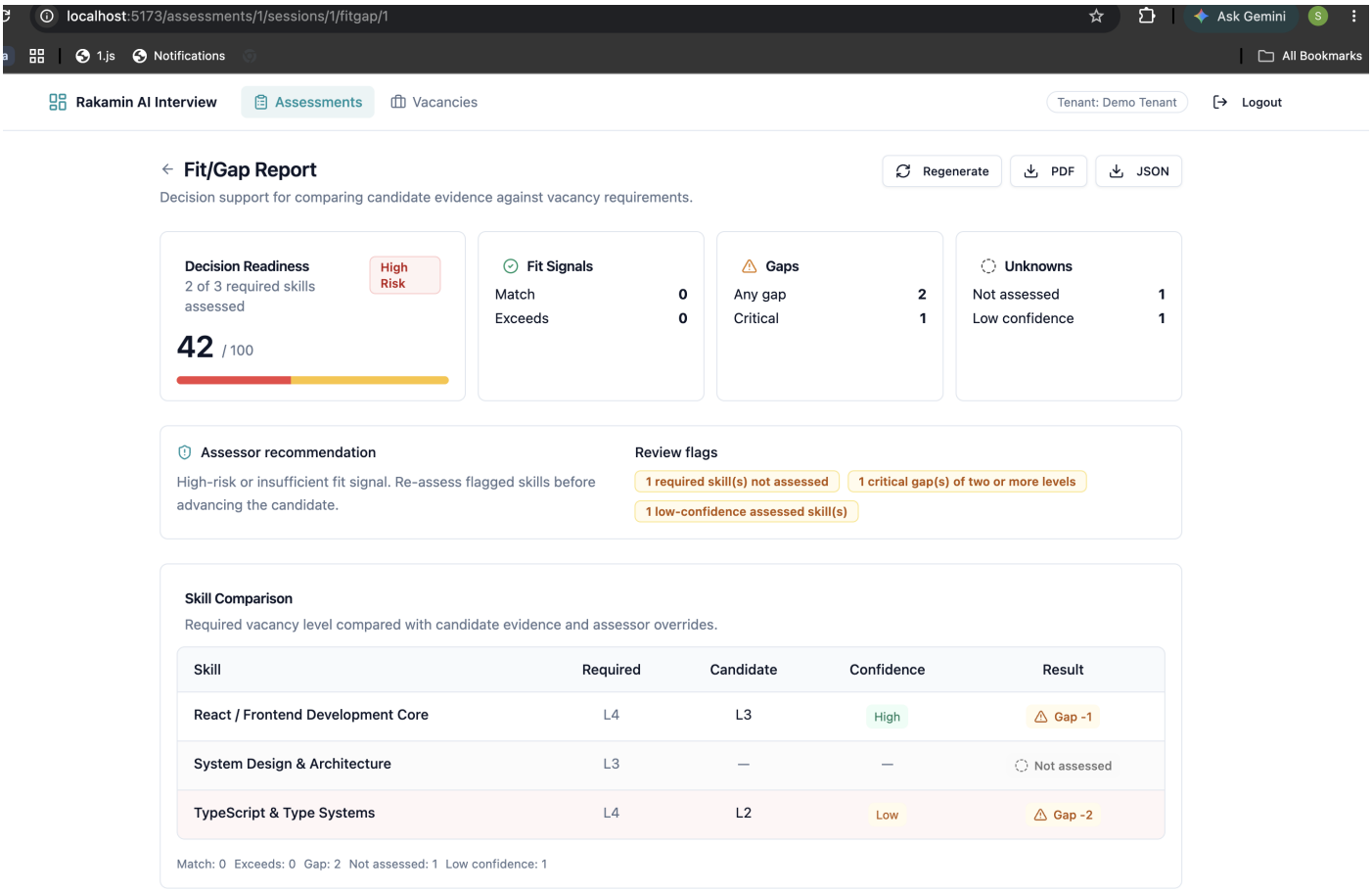
```
samudrazulqifli@samudras-MacBook-Air api % bundle exec rspec spec/service
s/fit_gap/summary_spec.rb
...

Finished in 0.00432 seconds (files took 0.05732 seconds to load)
3 examples, 0 failures
```

**Figure 2. Frontend production build success**



## Figure 3. Fit/Gap dashboard desktop with risk summary and skill comparison



# Figure 4. Culture narrative and discovered skills

